Laboratorio 2 · Lakehouse end-to-end en AWS: batch y streaming

SI7006 · Almacenamiento y Procesamiento de Grandes Datos · EAFIT 2026-2

Estudiante: David Lopera Londoño

En este laboratorio se construyó el lakehouse de PayTrack de punta a punta. Los datos llegan de dos formas, y cada una tiene su parte:

- **Parte A · Batch:** los lotes diarios pasan por la arquitectura Medallion (Bronze → Silver → Gold) y quedan en tablas Delta Lake.
- **Parte B · Streaming:** los pagos llegan en tiempo real, se procesan con Spark Structured Streaming y terminan en las mismas tablas del batch.

Todo se desarrolló primero en local, gratis (Docker con Spark y Delta Lake, y Kafka como broker), y al final se ejecutó en AWS: S3, Glue 5.0, Kinesis y Athena, desplegados como infraestructura como código con Serverless Framework. El código de los jobs es exactamente el mismo en los dos entornos; solo cambia dónde viven los datos y de dónde llega el streaming. La Figura 1 (página 2) resume la arquitectura; las capturas de la ejecución se entregan aparte.

1. Ejecución y resultados generales

Los dos lotes traen errores a propósito: duplicados, montos inválidos y clientes faltantes. La Tabla 1 muestra qué hizo el pipeline con cada uno; los números fueron idénticos en local y en AWS.

Lote	Filas en raw	Duplicados	Transacciones distintas	Nuevas en silver	Actualizadas	Cuarentena
Lote 1 (2026-09-29)	30.600	600	30.000	28.239	0	1.761 (5,9 %)
Lote 2 (2026-09-30)	2.316	40	2.276	1.884	276	116 (5,1 %)

Tabla 1. Resultado del pipeline batch por lote.

Todas las validaciones de los jobs quedaron en verde. En AWS, los 8 jobs batch (4 por lote) terminaron en Succeeded en unos 2 minutos cada uno, y casi todo ese tiempo fue Glue arrancando; el job de streaming corrió 7 minutos y 11 segundos.

1.1 Un ajuste necesario en el streaming

El primer intento del job de streaming en AWS falló con KeyError: '__src_timestamp'. La función read_events de common.py espera que el conector de Kinesis entregue la hora de llegada de cada registro en esa columna, como dice la documentación, pero en Glue 5.0 llegó con otro nombre: addrecord_timestamp$_temporary$.

- Se ajustó read_events para buscar la columna disponible y, si no la encuentra, usar la hora de procesamiento.
- No afecta ningún resultado: esa columna solo alimenta ingest_date y un desempate, mientras que el watermark y las ventanas usan event_time.

Es justo el tipo de problema que no aparece en local, y por eso vale la pena correr también en la nube.

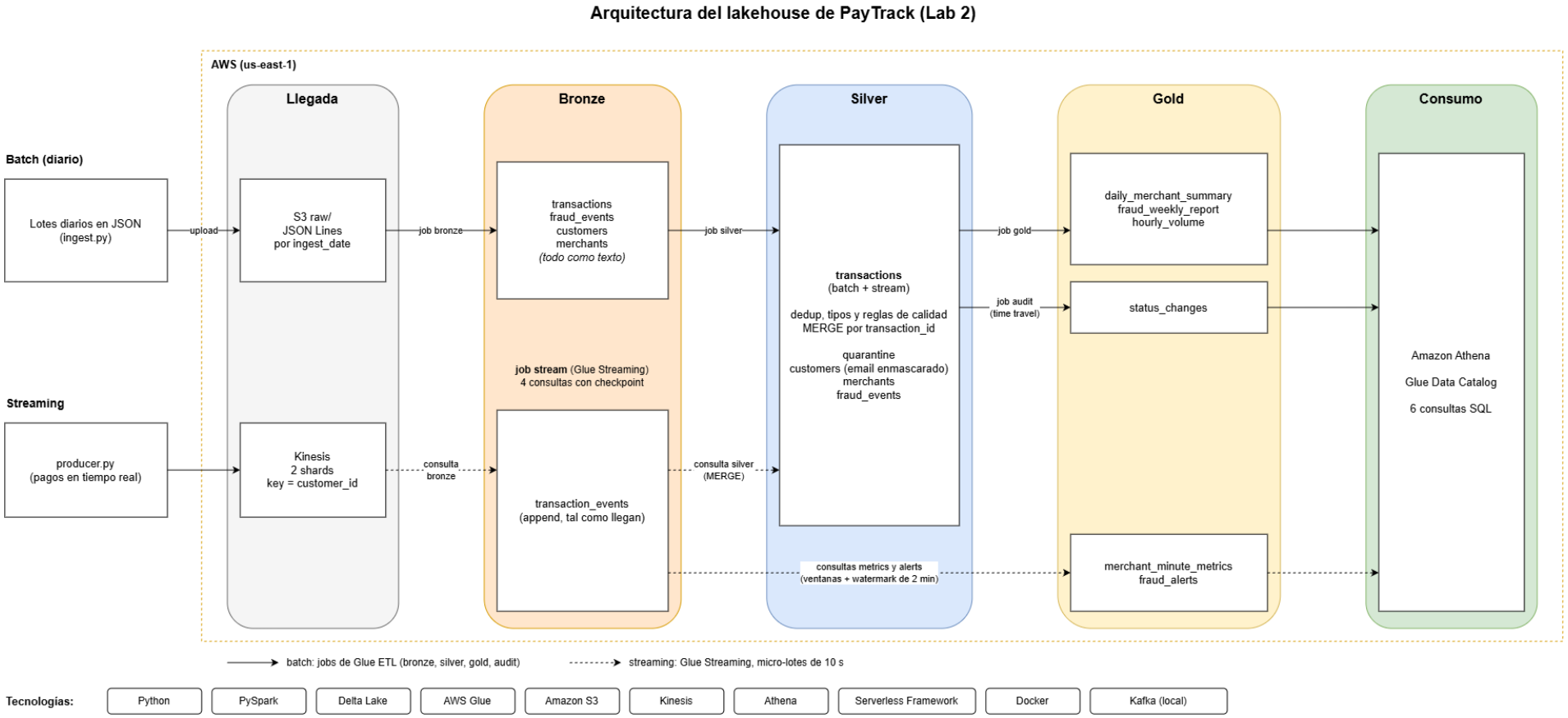


Figura 1. Arquitectura del lakehouse: capas, tablas, jobs batch, consultas de streaming y servicios de AWS.

2. Parte A · Batch

2.1 Bronze: todo como texto y replaceWhere (pregunta 1)

Bronze guarda cada lote tal como llegó, pasando de JSON a Parquet pero **leyendo todas las columnas como texto**. Si un lote trae un monto escrito como "N/A" y Bronze intentara leerlo como número, el job fallaría o Spark inferiría un tipo distinto al del lote anterior, y la tabla quedaría con esquemas incompatibles. Como texto todo entra siempre; decidir qué es válido le toca a Silver.

La escritura usa **replaceWhere** sobre la partición ingest_date:

- Con **append**, re-ejecutar un lote lo agrega otra vez y quedan duplicados.
- Con un **overwrite** normal, se borrarían los lotes anteriores.
- Con **replaceWhere**, solo se reemplaza la partición de ese lote, de forma atómica.

La prueba de idempotencia lo confirma: al ejecutar el lote 2 dos veces, Bronze siguió con 2.316 filas para ese lote (con append habrían sido 4.632) y la auditoría reportó 0 insertadas y 0 actualizadas.

2.2 Silver: cuarentena y orden de los pasos (pregunta 2)

Silver deja una fila por transacción, con tipos correctos y reglas de calidad. Lo que no pasa las reglas va a una tabla de cuarentena con el motivo, así no se pierde nada. La Tabla 2 muestra cuántas filas fueron a cuarentena en cada lote.

Motivo (dq_reason)	Lote 1	Lote 2
missing_customer	591	39
invalid_amount	1.170	77
invalid_status	0	0
Total	**1.761**	**116**

Tabla 2. Transacciones en cuarentena por motivo (se guarda el primer problema encontrado).

- Dos de cada tres rechazos son **montos inválidos** (nulos, no numéricos, menores o iguales a cero, o de 100 millones o más).
- No hubo **estados inválidos**, porque el generador no inyecta ese error. Aun así la regla revisa los nulos de forma explícita: NOT isin(...) sobre un NULL da NULL, y sin ese cuidado la fila pasaría como válida.

Se deduplica antes de validar porque, si no, cada copia de una fila mala iría otra vez a la cuarentena y dejaría de cuadrar la validación "válidas + cuarentena = transacciones distintas del lote". Además, la deduplicación se queda con la versión más reciente de cada transacción, que es la única que vale la pena evaluar.

2.3 MERGE: actualizar solo si cambió el status (pregunta 3)

Cada lote se integra a Silver con un MERGE: las transacciones nuevas se insertan y las que ya existen se actualizan **solo si su status cambió**. Ese es el caso de los 276 contracargos del lote 2, que pasan de approved a chargeback.

Sin esa condición, re-ejecutar un lote "actualizaría" las 2.160 transacciones que ya existen aunque nada hubiera cambiado. silver_updated_at cambiaría, el MERGE reportaría 2.160 filas actualizadas y la auditoría fallaría, porque encontraría 0 cambios reales de estado.

El historial de la tabla mostró además un detalle interesante: la condición garantiza que **los datos** no cambien, pero no que **los archivos** no se reescriban. En la re-ejecución, el MERGE no tocó ningún dato y aun así reescribió las 30.123 filas de la tabla (numTargetRowsCopied). Delta reescribe todos los archivos donde coincide la llave, no solo donde cambia el status. Se evitaría filtrando la fuente antes del MERGE, para dejar solo las filas nuevas o con un estado distinto.

2.4 Auditoría con time travel (pregunta 4)

Cada MERGE crea una versión nueva de la tabla Delta, y las versiones anteriores se pueden seguir leyendo. El job de auditoría aprovecha eso: compara la versión 1 de silver.transactions (antes del lote 2) con la versión 2 (después) y saca las transacciones que cambiaron de estado, sin haber guardado ninguna copia.

El resultado: **276 contracargos que revierten $19.851.503,02 COP**. Cuadra exactamente con las métricas del MERGE (1.884 insertadas y 276 actualizadas) y con el TODO 4 de Athena, que los reparte por categoría (Tabla 3). Retail concentra el 27 % del monto.

Categoría	Contracargos	Monto revertido (COP)
retail	61	5.383.780,86
travel	38	3.055.249,30
telco	36	2.586.766,12
grocery	38	2.520.992,74
streaming	32	2.430.057,37
transport	40	1.961.137,93
delivery	31	1.913.518,70
Total	**276**	**19.851.503,02**

Tabla 3. Contracargos del lote 2 por categoría de comercio.

¿Y después de un VACUUM? VACUUM borra los archivos que la versión actual ya no usa y que son más viejos que el periodo de retención (7 días por defecto). La versión 1 depende de archivos que el MERGE reemplazó, así que dejaría de poder leerse y la auditoría fallaría. El time travel no es un respaldo permanente: por eso el resultado se guarda en gold.status_changes, y para auditar a largo plazo convendría activar el Change Data Feed de Delta.

2.5 PII y cambios de los clientes (pregunta 5)

silver.customers se actualiza como **SCD tipo 1**: si un cliente cambia de ciudad o de segmento, el valor anterior se sobrescribe. Lo que se pierde es poder analizar con el segmento que el cliente tenía en el momento de cada transacción. Por ejemplo, un fraude cometido cuando el cliente era mass se contaría como premium si después cambió de segmento.

Es el mismo problema de la consulta Q2 en el Lab 1, y se resuelve igual: con **SCD tipo 2**, guardando una fila por cada versión del cliente con su periodo de vigencia (valid_from, valid_to, is_current). En Delta se haría con un MERGE que cierra la versión vigente cuando cambian la ciudad o el segmento e inserta la nueva, y cada transacción se uniría con la versión vigente en su fecha. En cuanto a la protección de datos, el email queda enmascarado (car***@gmail.com) y el nombre se elimina, como pide la Ley 1581 de 2012.

2.6 Consumo: Gold vs. Silver (pregunta 6)

Para ver para qué sirve Gold, se respondió la misma pregunta (los 5 comercios con más monto aprobado en septiembre) desde la tabla Gold y desde Silver.

Consulta	Data scanned	M004 aprobado (COP)	M004 tasa	4.º y 5.º lugar
TODO 1 · Gold, antes de recalcular	3,26 KB	152.573.696,30	98,98 %	M007, M003
TODO 1 · Gold, recalculado	3,35 KB	151.809.363,99	98,42 %	M003, M007
TODO 2 · Silver	87,73 KB	151.809.363,99	98,42 %	M003, M007

Tabla 4. Top 5 de comercios de septiembre desde Gold y desde Silver (Athena).

Gold escanea unas 27 veces menos datos, porque lee filas ya agregadas (una por día y comercio) en lugar de recorrer todas las transacciones del mes. En Athena, que cobra por dato escaneado, eso es directamente menos costo y menos tiempo.

Lo interesante es que la primera vez los resultados **no coincidían**. El streaming había aplicado contracargos sobre transacciones de septiembre después de que corriera el job Gold, así que Gold seguía mostrando esos montos como aprobados. Al recalcular Gold, todo quedó idéntico. Eso muestra lo que se pierde con Gold: es una foto que hay que refrescar, y solo responde las preguntas para las que fue diseñado (por ejemplo, no tiene el método de pago). La tasa mensual se calculó ponderando las tasas diarias por el número de transacciones, no promediándolas.

El TODO 3, por su parte, muestra que la semana del 3 de agosto fue la de más fraude para los segmentos mass (12 eventos) y high_value (6), y la del 24 de agosto para premium (4).

3. Parte B · Streaming

El job de streaming corre cuatro consultas al mismo tiempo, cada una con su propio checkpoint para saber hasta dónde leyó:

- **bronze:** guarda cada evento tal como llega del broker.
- **silver:** integra los eventos a la misma silver.transactions del batch, con un MERGE por micro-lote.
- **metrics:** calcula métricas por comercio en ventanas fijas de 1 minuto.
- **alerts:** busca clientes con 6 o más pagos en ventanas de 30 s que avanzan cada 10 s.

El productor simula los problemas de una fuente real: reenvía eventos, manda algunos con varios minutos de retraso y genera ráfagas de pagos fraudulentos. La Tabla 5 resume las ejecuciones.

Ejecución	Eventos leídos	Descartados por watermark	Transacciones nuevas en silver	Ventanas de métricas	Alertas
Local · availableNow (--fast)	12.260	4 (0,03 %)	11.633	94	103
Local · en vivo, 20 ev/s	2.428	39 (1,6 %)	2.309	34	32
Local · en vivo, 50 ev/s	6.118	95 (1,6 %)	5.817	53	19 *
AWS · en vivo (Kinesis)	9.847	50 (0,5 %)	9.370	106	104

*Tabla 5. Ejecuciones del streaming. * Ventanas de la corrida anterior; de la corrida a 50 ev/s no se emitió ninguna.*

3.1 Garantías de entrega (pregunta 7)

Hay que mirar la garantía en cada tramo:

- **Productor de Kafka:** con acks=all y enable.idempotence, un evento confirmado no se pierde aunque falle el líder, y los reintentos no duplican. Entre el productor y el broker es **exactly-once**.
- **Kinesis con reintentos:** es **at-least-once**. Si un put_records sí escribió pero la respuesta se perdió, el reintento duplica el registro.
- **La fuente en sí** siempre es at-least-once, porque el productor reenvía eventos a propósito.

Aun así, las tablas quedan como si cada evento se hubiera procesado exactamente una vez. Lo logra la combinación de cuatro piezas: el checkpoint recuerda hasta dónde leyó cada consulta, la escritura de cada micro-lote en Delta es atómica, la deduplicación por event_id elimina los reenvíos y el MERGE deja el mismo resultado si un micro-lote se repite. La prueba de idempotencia lo demuestra: al relanzar el job sin eventos nuevos, las cuatro consultas leyeron 0 filas y ninguna tabla cambió.

3.2 Watermark (pregunta 8, experimento 1)

El watermark le dice a Spark cuánto esperar los eventos atrasados (aquí, 2 minutos) para poder cerrar ventanas y olvidar estado viejo. El productor envía la misma proporción de eventos tardíos en todos los modos (3 %, con retrasos de 30 s a 6 min), pero **en vivo se descartaron unas 50 veces más** que en modo availableNow (1,6 % frente a 0,03 %).

La razón es que el watermark solo avanza **entre micro-lotes**. Con --fast, los 10 minutos de eventos ya estaban en Kafka y se procesaron en solo 5 micro-lotes grandes: los eventos tardíos llegaban junto con los demás, antes de que el watermark avanzara. En vivo hay un micro-lote cada 10 segundos, el watermark sigue el ritmo del tiempo real y un evento con más de 2 minutos de retraso encuentra su ventana ya cerrada.

Por eso **la consulta silver no usa watermark**: un pago tardío sigue siendo un pago real. Si lo usara, en la prueba en vivo se habría perdido cerca del 1,6 % de los pagos.

3.3 Latencia de las alertas (pregunta 9)

Las alertas se escriben en modo append, que publica cada ventana una sola vez, cuando el watermark la cierra. Eso pasa cuando llega un evento al menos 2 minutos posterior al fin de la ventana, más la espera del siguiente micro-lote. En la práctica, una alerta aparece **entre 2 y 2,7 minutos** después del último pago de la ráfaga, y casi todo ese tiempo se debe al watermark y al modo append.

La prueba a 50 eventos/s lo mostró de forma muy clara: el envío duró 2 minutos, lo mismo que el watermark, así que **ninguna ventana de esa corrida alcanzó a cerrarse** y no se emitió ni una alerta. Las 19 alertas que aparecieron eran ventanas de la prueba a 20 ev/s de dos horas antes, que se habían quedado abiertas y solo se

publicaron cuando los eventos nuevos movieron el watermark. Para bajar la latencia se podría usar un watermark más corto (a cambio de perder más eventos tardíos) o el modo update con foreachBatch y MERGE, que publica la alerta apenas el conteo llega a 6.

3.4 **Precisión de las alertas** (pregunta 10, experimento 2)

Una alerta es un acierto si el cliente de verdad hizo un pago fraudulento en esa ventana. Se midió a 20 eventos/s en AWS y en local, y a 50 eventos/s en local. Como a 50 ev/s el streaming no alcanzó a emitir alertas, para esa corrida se aplicó la misma regla en batch sobre los pagos de Bronze. A 20 ev/s ese cálculo dio exactamente las mismas 154 alertas que emitió el streaming, lo que confirma que la consulta está bien implementada.

Corrida	Alertas	Aciertos	Precisión
AWS · 20 ev/s (TODO 5)	104	87	83,65 %
Local · 20 ev/s	154	127	82,47 %
Local · 50 ev/s (regla en batch)	392	62	15,82 %

Tabla 6. Precisión de las alertas de ráfagas según la carga.

A 50 ev/s la precisión se desploma, y la razón es estadística. Los pagos se reparten entre los mismos 500 clientes: a 20 ev/s cada cliente hace en promedio 1,2 pagos por ventana de 30 s, y a 50 ev/s hace 2,6. Con una distribución de Poisson, la probabilidad de que un cliente normal llegue a 6 pagos por azar sube de 0,15 % a cerca de 5 %, lo que da unas 350 falsas alarmas en la corrida; se observaron 330. Las ráfagas reales se siguen detectando, pero quedan escondidas entre el ruido. Para mejorarla habría que usar un umbral relativo a la carga o al comportamiento normal de cada cliente, combinarlo con otras señales como el monto, y consolidar las alertas que se solapan (hoy una misma ráfaga genera 2 o 3).

3.5 **Contracargo como estado final** (pregunta 11)

En el MERGE del streaming, un contracargo no se puede revertir. El escenario que lo justifica es el reenvío: el productor repite eventos que ya envió, y puede reenviar el pago original de una transacción **después** de su contracargo. Si ese reenvío llega en un micro-lote posterior, latest_per_transaction solo ve ese pago con status approved, y sin la condición el MERGE cambiaría chargeback por approved.

latest_per_transaction no basta porque decide **dentro de un micro-lote**, mientras que el estado anterior vive en la tabla. Por eso la regla tiene que estar en el MERGE, que sí compara contra lo que ya está guardado.

3.6 **Partition key** (pregunta 12, experimento 3)

Kafka y Kinesis reparten los eventos entre particiones según su llave, y solo garantizan el orden dentro de una partición. Se eligió **customer_id** para que todos los eventos de un cliente (pagos, ráfagas y contracargos) lleguen juntos y en orden. Para el experimento se enviaron eventos con otras dos llaves a topics aparte y se comparó el reparto (Tabla 7).

Partition key	Partición 0	Partición 1	Partición 2	Total
Constante ("x")	1.222 (100 %)	0	0	1.222
transaction_id	412 (33 %)	404 (33 %)	420 (34 %)	1.236
customer_id (la elegida)	5.571 (38 %)	4.333 (29 %)	4.784 (33 %)	14.688

Tabla 7. Reparto de los eventos entre las 3 particiones de Kafka según la partition key.

- **Constante:** todo cae en una partición. Se pierde el paralelismo y, en Kinesis, ese shard se satura a los 1.000 registros por segundo.
- **transaction_id:** el reparto es casi perfecto, pero los pagos de una ráfaga quedan en particiones distintas.
- **customer_id:** el reparto es aceptable pero menos parejo, porque solo hay 500 llaves y algunos clientes envían más eventos, sobre todo en las ráfagas.

El número de alertas **no cambiaría** con transaction_id, porque Spark reagrupa los eventos por customer_id antes de contar, sin importar de qué partición vinieron. La partition key importa por el orden: es lo que garantiza que un contracargo llegue después de su pago para cualquier consumidor que lea la partición en secuencia.

3.7 Batch y streaming en la misma tabla (pregunta 13)

Cada MERGE crea una versión de silver.transactions. Antes de la prueba a 50 ev/s, la tabla local tenía 19 versiones (4 del batch y 15 del streaming), y esa prueba agregó 7 más. El historial muestra el costo real de reescribir:

- El MERGE del lote 2 copió **27.963 filas** para actualizar 276 contracargos.
- Los 15 MERGE del streaming insertaron 13.942 transacciones y actualizaron 273, pero copiaron **380.124 filas sin cambios**, unas 27 por cada fila que de verdad cambió. Un micro-lote con una sola actualización reescribió 15.145 filas.

Con 500K pagos diarios esto se vuelve un problema de **amplificación de escritura**: cada 10 segundos se reescriben archivos completos y se acumulan versiones y archivos pequeños. Se resolvería agregando year y month a la condición del MERGE para que Delta descarte particiones, activando deletion vectors (marcan las filas cambiadas sin reescribir el archivo), compactando con OPTIMIZE y Z-ORDER, y alargando el trigger.

Si el batch y el stream escribieran al mismo tiempo, Delta resuelve la concurrencia de forma optimista: el segundo commit detecta que el primero modificó los mismos archivos y falla con una excepción de conflicto. Habría que reintentarlo, separar por partición lo que escribe cada uno o pausar el stream mientras corre el batch.

Finalmente, el TODO 6 de Athena compara la capa rápida con la batch: las métricas por minuto cuentan entre 301 y 347 pagos menos por comercio que Silver (6.799 frente a 9.370 en total). Solo 50 se deben al watermark; el resto son las ventanas de los últimos 2 a 3 minutos, que nunca se cerraron porque el productor dejó de enviar. **La fuente de verdad es Silver**: la capa rápida sirve para monitorear en vivo, no para conciliar.

4. Costos y operación (pregunta 14)

Con 500K transacciones diarias (unos 6 eventos por segundo en promedio) y los precios de lista en us-east-1, el costo mensual se reparte así:

Servicio	Supuesto	USD / mes
S3	~10 GB nuevos al mes (raw en JSON + tablas Delta comprimidas)	< 3
Glue batch	4 jobs diarios × 2 DPU × ~5 min, a $0,44 por DPU-hora	~9
Glue Streaming 24/7	2 DPU × 730 h, a $0,44 por DPU-hora	~642
Kinesis	2 shards × 730 h + 15 millones de PUT	~22
Athena	~1 TB escaneado al mes, a $5 por TB	~5
Total		**~680**

Tabla 8. Costo mensual estimado con 500K transacciones diarias.

Casi todo el costo (94 %) es Glue Streaming encendido 24/7, y 6 eventos por segundo no necesitan 2 DPU todo el tiempo. Si el negocio no necesitara alertas casi en tiempo real, lo más simple sería lanzar el mismo job en modo availableNow cada hora con un disparador programado: procesa lo que acumuló Kinesis y termina, y los checkpoints garantizan que no se pierde ni se repite nada. Costaría unos $30 al mes, con hasta una hora de latencia. Si sí se necesita 24/7, se puede bajar con workers G.025X, pensados para streams de bajo volumen (unos $160 al mes), con Glue 6.0, que cobra 30 % menos, y con un solo shard de Kinesis.

Para la zona raw se definió una regla de ciclo de vida que pasa los lotes a STANDARD_IA a los 30 días y a GLACIER_IR a los 90, sin expiración, porque es la única copia original y los soportes financieros se deben conservar por años. **En lakehouse/ nunca se debe poner expiración**: S3 borraría archivos Parquet sin que el _delta_log se entere, y la tabla quedaría apuntando a archivos que no existen. Las tablas Delta se limpian con VACUUM. Al terminar el laboratorio se eliminaron todos los recursos y se verificó en S3, CloudFormation y Kinesis.

Conclusión

El lakehouse procesa los dos tipos de datos sobre las mismas tablas y llega a los mismos resultados en local y en AWS: los 276 contracargos y los $19,85 millones revertidos aparecieron igual en la auditoría local y en Athena. Lo que lo hace confiable no es una sola pieza, sino la combinación de replaceWhere en Bronze, el MERGE condicional en Silver, y los checkpoints y la deduplicación en el streaming: cualquier paso se puede repetir sin alterar el resultado.

Las capas derivadas, Gold y la capa rápida, son más baratas y rápidas de consultar, pero se desactualizan o quedan incompletas; la fuente de verdad es Silver. A escala, el costo está en dos lugares: mantener el streaming encendido todo el tiempo y reescribir archivos en cada MERGE, y ahí estaría el siguiente paso de optimización.